

Course

« *Deep Learning* »

Sid-Ahmed Berrani

ENSIA

2024 – 2025

Sequence models

- Examples of sequence data

Speech recognition



→ « Ne demandez jamais quelle est l'origine d'un homme ; interrogez plutôt sa vie, son courage, ses qualités et vous saurez qui il est. »

Music generation

Random number



Sentiment classification

Nothing interesting in this book



DNA sequence analysis

Machine translation

Video activity analysis



→ Running

Name entity recognition

Mary from the HR department said that The Ritz London was a great hotel option to stay in London.

→ Mary from the HR department said that The Ritz London was a great hotel option to stay in London.

Sequence models: Notation

- Example of name entity extraction application

x: Mary from the HR department said that The Ritz London was a great hotel option to stay in London.

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$ $x^{<4>}$ $x^{<5>}$... $x^{<17>}$ $x^{<18>}$ $x^{<19>}$

The length of the input sequence: $T_x = 19$

y: 1 0 0 0 0 ... 0 0 1

$y^{<1>}$ $y^{<2>}$ $y^{<3>}$ $y^{<4>}$ $y^{<5>}$... $y^{<17>}$ $y^{<18>}$ $y^{<19>}$

The length of the output sequence: $T_y = 19$

$\mathbf{X}^{(i)}$: the i^{th} input sequence

$\mathbf{X}^{(i)<t>}$: the t^{th} element of the i^{th} input sequence

$T_x^{(i)}$: the length of i^{th} input sequence

$\mathbf{Y}^{(i)}$: the i^{th} output sequence

$\mathbf{Y}^{(i)<t>}$: the t^{th} element of the i^{th} output sequence

$T_y^{(i)}$: the length of i^{th} output sequence

Sequence models: Notation

- Words representation

=> We need a vocabulary (also called a dictionary).

Mary from the HR department said that The Ritz London was a great hotel option to stay in London.

$x^{<1>}$ $x^{<2>}$ $x^{<3>}$ $x^{<4>}$ $x^{<5>}$

... $x^{<17>}$

$x^{<17>}$ $x^{<18>}$ $x^{<19>}$

Vocabulary

$\begin{bmatrix} a \\ \vdots \\ from \\ \vdots \\ stay \\ \vdots \\ Zyzzyva \end{bmatrix}$

A 10,000
dictionary

$\downarrow$
 $\begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \\ \vdots \\ 0 \end{bmatrix}$

$\downarrow$
 $\begin{bmatrix} 1 \\ \vdots \\ 0 \\ \vdots \\ 0 \\ \vdots \\ 0 \end{bmatrix}$

$\downarrow$
 $\begin{bmatrix} 0 \\ \vdots \\ 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}$

The one-hot representation

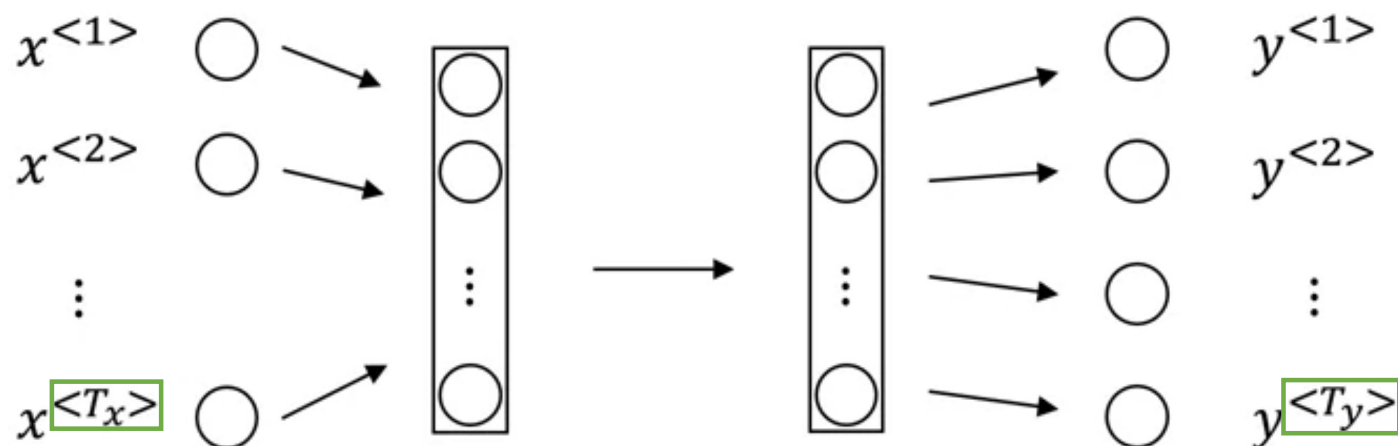
Sequence models: Notation

Words representation

- In modern NLP, the size of dictionary can be 100,000
- Internet companies use dictionaries of 1,000,000 words.
- For commercial use, a dictionary of 30,000 to 50,000 words is very common.
- The dictionary can be extracted for the training dataset: The 10,000 most frequent words.
- To cope with out-of-vocabulary words, we use a special token <UNKOWN>. It is considered as a *fake* word in the vocabulary

Sequence models

- Why a standard NN cannot be used for sequences?



Problems:

- Inputs and outputs can be of different lengths in different examples.
- The input layer may be very large: $10,000 \times T_x$
- The context is not taken into account: It does not share features learned across different positions.

Sequence models: Recurrent NN

- Definition:

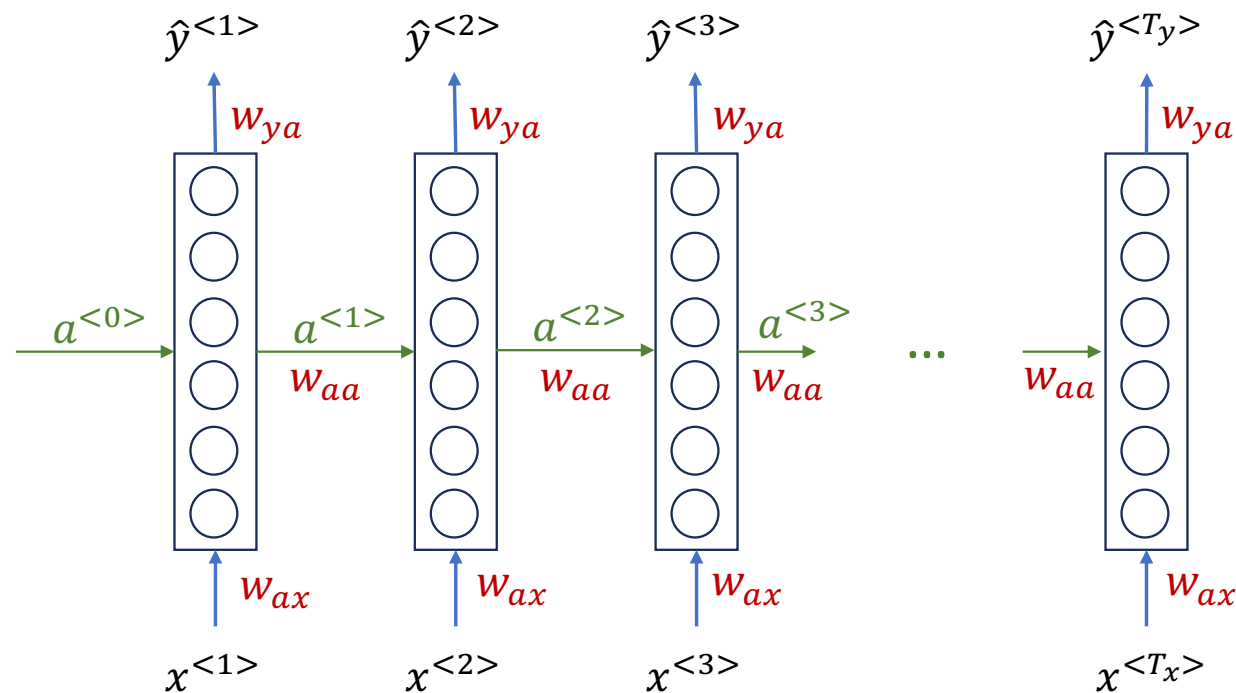
A type of artificial neural network designed for modeling sequential data by capturing temporal dependencies within the data.

A neural network architecture where connections between units form a directed cycle, allowing the network to maintain a memory of previous inputs.

At each time step t , the network processes an input vector x_t and updates a hidden state h_t , which encodes information about the current input and the past sequence.

Sequence models: Recurrent NN

- Parameters are also shared!

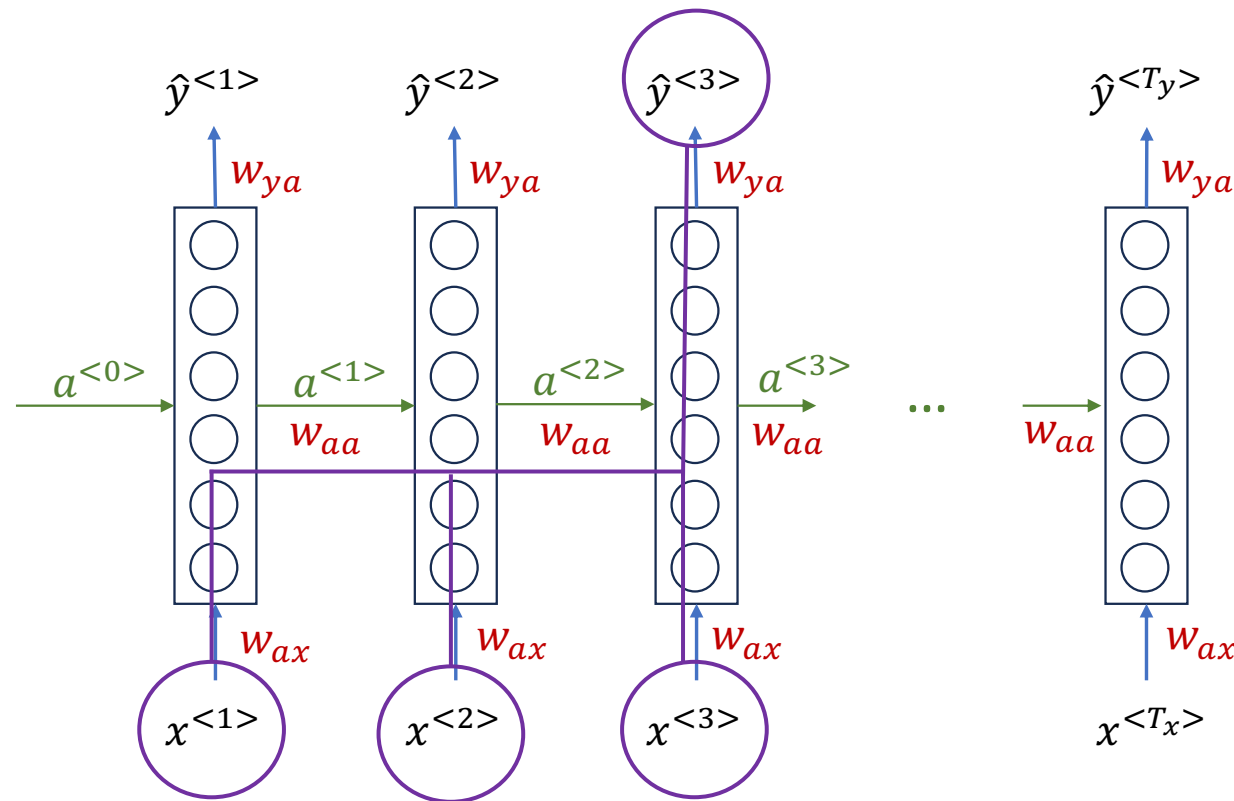


Parameters:

- w_{ax} : input $\rightarrow$ hidden layer
- w_{aa} : horizontal connections
- w_{ya} : hidden $\rightarrow$ output predictions

Sequence models: Recurrent NN

- Parameters are also shared!

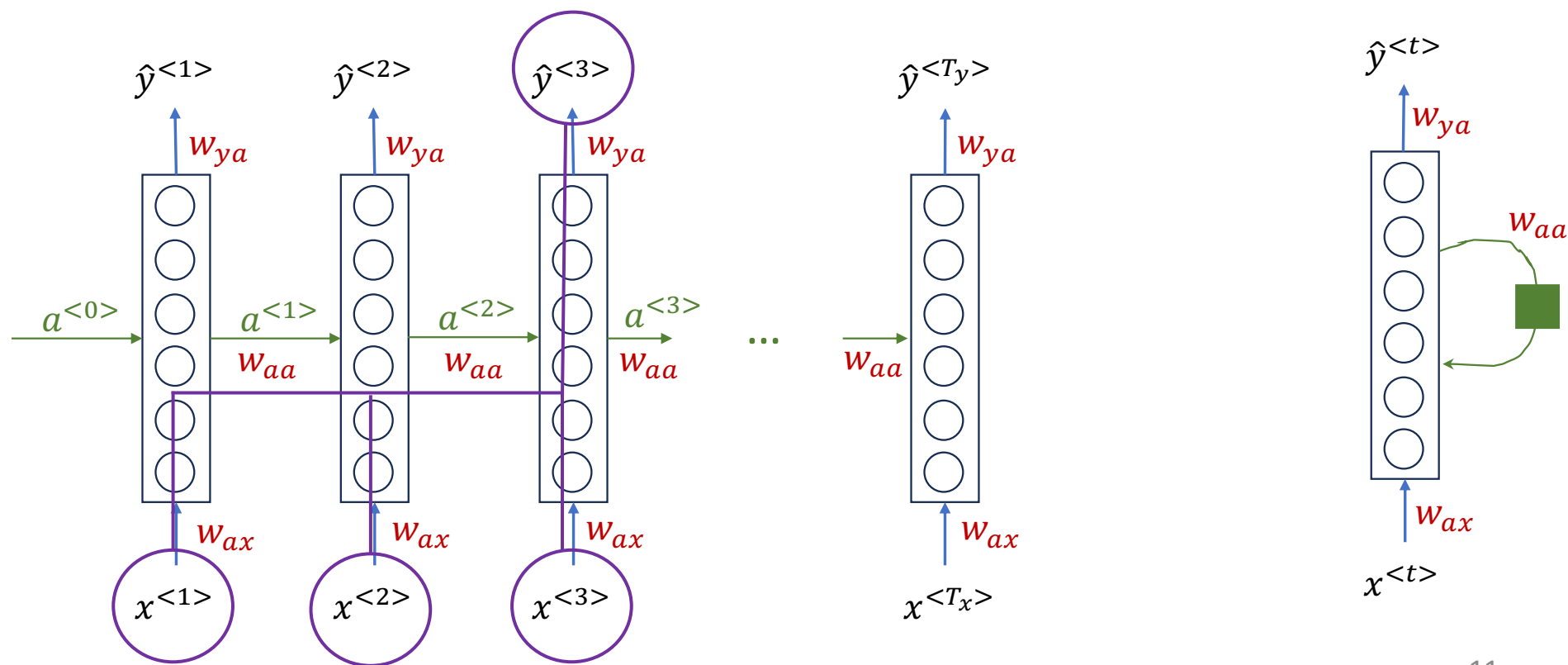


Parameters:

- w_{ax} : input $\rightarrow$ hidden layer
- w_{aa} : horizontal connections
- w_{ya} : hidden $\rightarrow$ output predictions

Sequence models: Recurrent NN

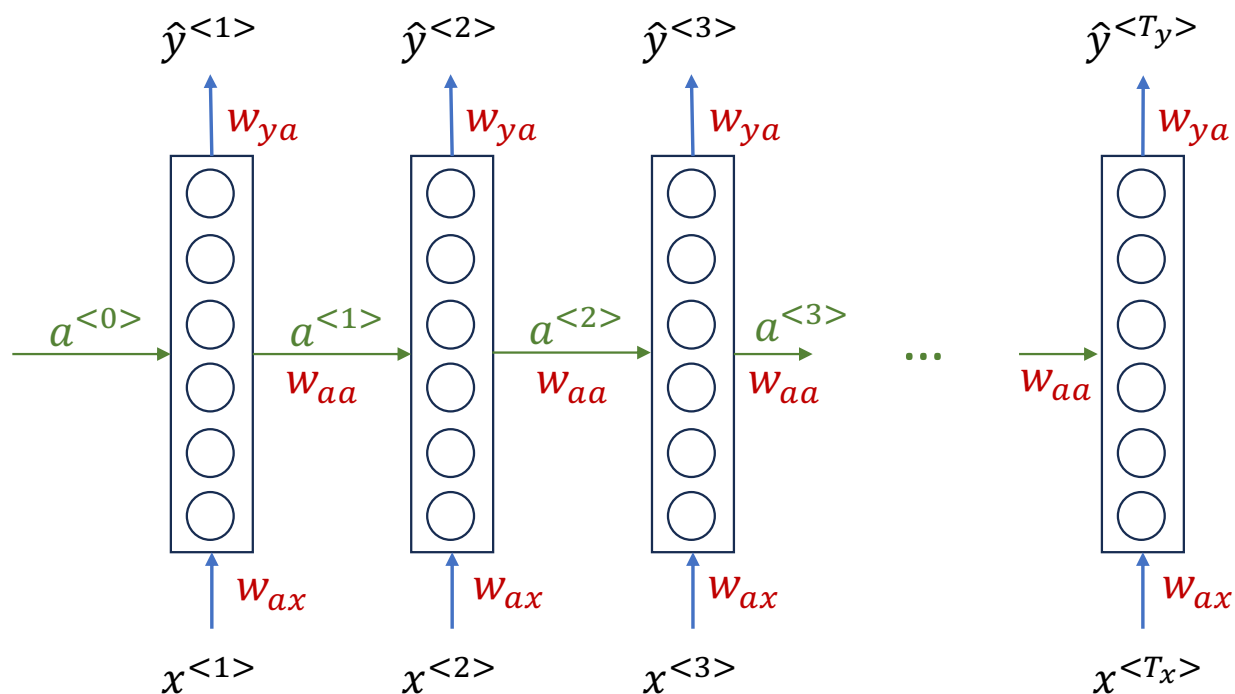
- Parameters are also shared!



Sequence models: Recurrent NN

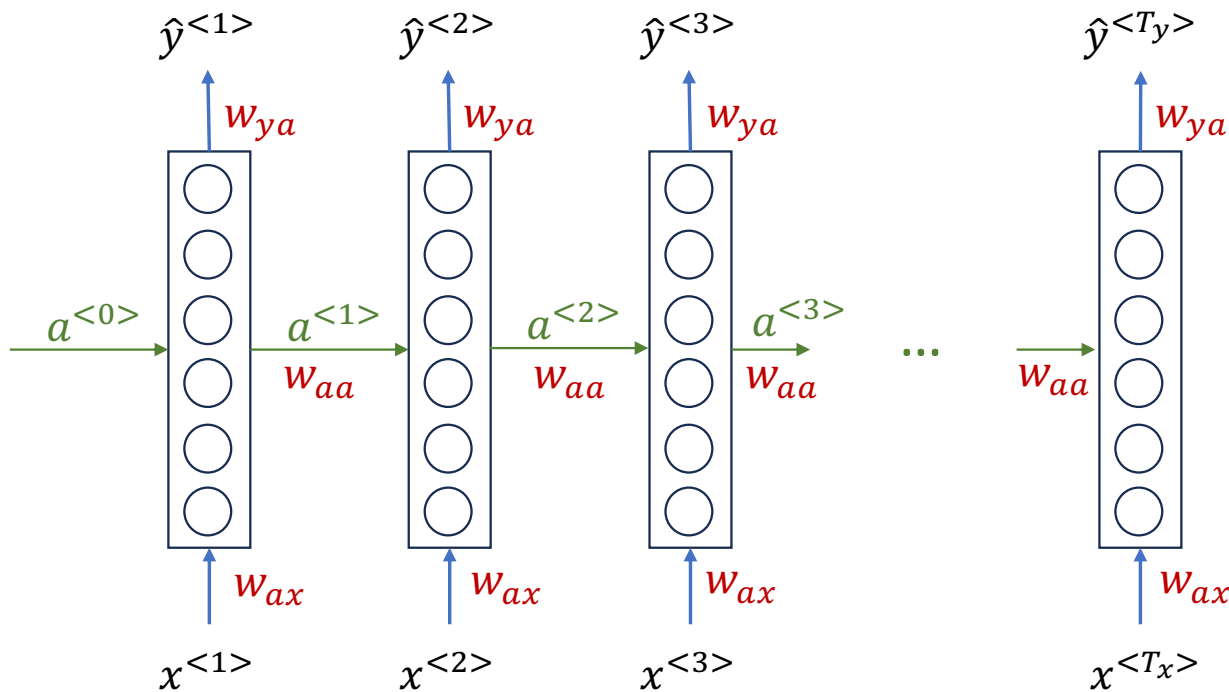
- Context in one direction: Only the past is taken into account!

To cope with this and take into account also the future => Bidirectional RNN



Sequence models: Recurrent NN

- Forward propagation



$a^{<0>} = a \text{ vector of zeros}$

$$a^{<1>} = g_1(w_{aa} \cdot a^{<0>} + w_{ax} \cdot x^{<1>} + b_a)$$

g_1 : The activation function (tanh / ReLU)

In RNN, tanh is generally used.

$$\hat{y}^{<1>} = g_2(w_{ya} \cdot a^{<1>} + b_y)$$

g_2 : The activation function (sigmoid)

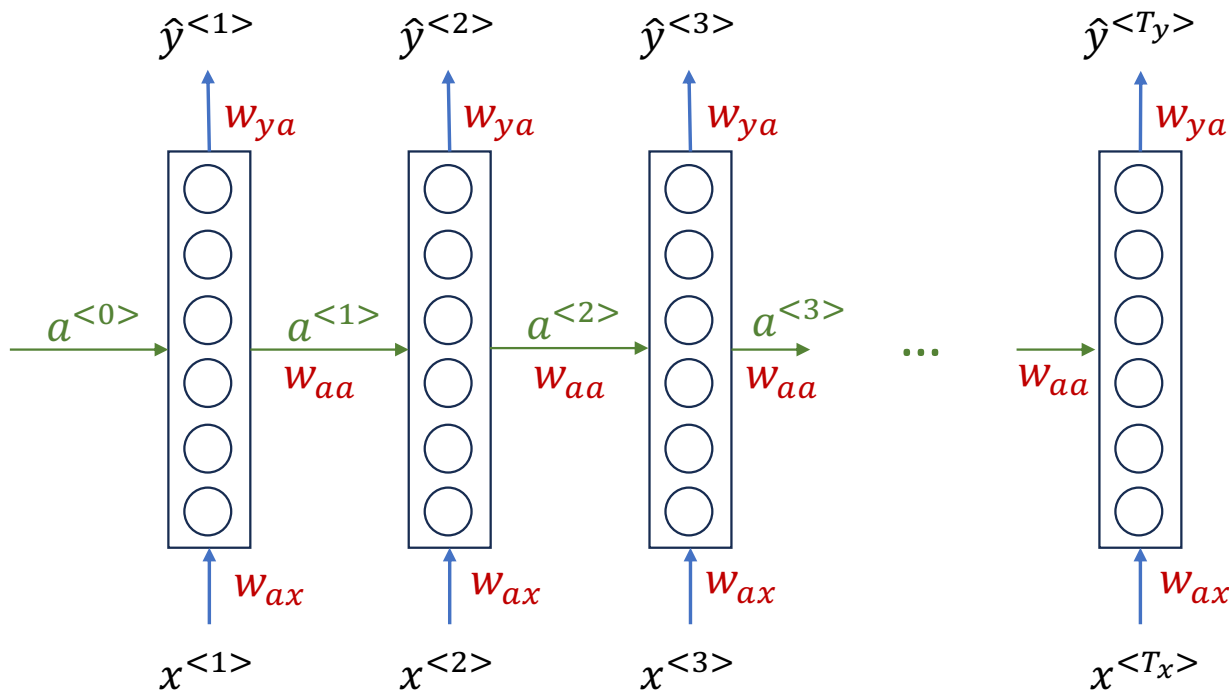
$$a^{<2>} = g_1(w_{aa} \cdot a^{<1>} + w_{ax} \cdot x^{<2>} + b_a)$$

$$\hat{y}^{<2>} = g_2(w_{ya} \cdot a^{<2>} + b_y)$$

...

Sequence models: Recurrent NN

- Forward propagation



$a^{<0>} = a \text{ vector of zeros}$

$$a^{<t>} = g_1(w_{aa} \cdot a^{<t-1>} + w_{ax} \cdot x^{<t>} + b_a)$$

$$\hat{y}^{<t>} = g_2(w_{ya} \cdot a^{<t>} + b_y)$$

Simplified notation:

$$a^{<t>} = g_1(w_a \cdot [a^{<t-1>}, x^{<t>}] + b_a)$$

$$\hat{y}^{<t>} = g_2(w_y \cdot a^{<t>} + b_y)$$

Sequence models: Recurrent NN

- Simplified notation

$$a^{<t>} = g_1(\underbrace{w_{aa}}_{50 \times 50} \cdot \underbrace{a^{<t-1>}}_{50 \times 1} + \underbrace{w_{ax}}_{50 \times 1000} \cdot \underbrace{x^{<t>}}_{1000 \times 1} + b_a)$$

$$a^{<t>} = g_1(\underbrace{w_a}_{50 \times 1050} \cdot \underbrace{[a^{<t-1>}, x^{<t>}]}_{1050 \times 1} + b_a)$$

$$w_a = [w_{aa} \vdots w_{ax}] \begin{matrix} \updownarrow 50 \\ \leftarrow 50 \quad \leftarrow 1000 \end{matrix}$$

$$[a^{<t-1>}, x^{<t>}] = \begin{bmatrix} a^{<t-1>} \\ x^{<t>} \end{bmatrix} \begin{matrix} \updownarrow 50 \\ \updownarrow 1000 \end{matrix}$$

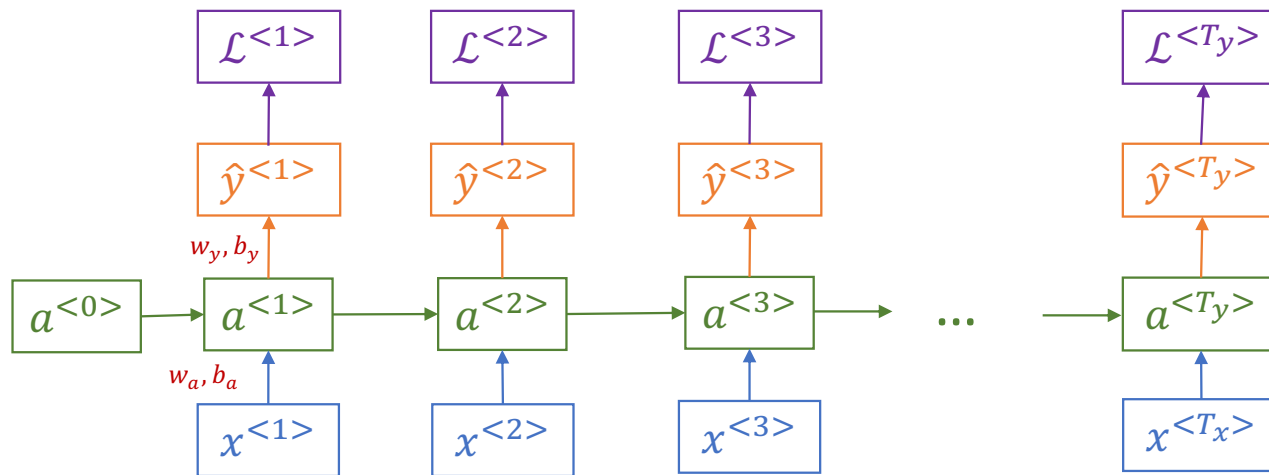
$$\hat{y}^{<t>} = g_2(w_{ya} \cdot a^{<t>} + b_y)$$

$$\hat{y}^{<t>} = g_2(w_y \cdot a^{<t>} + b_y)$$

Sequence models: RNN

- Forward propagation and backpropagation

$\mathcal{L}$



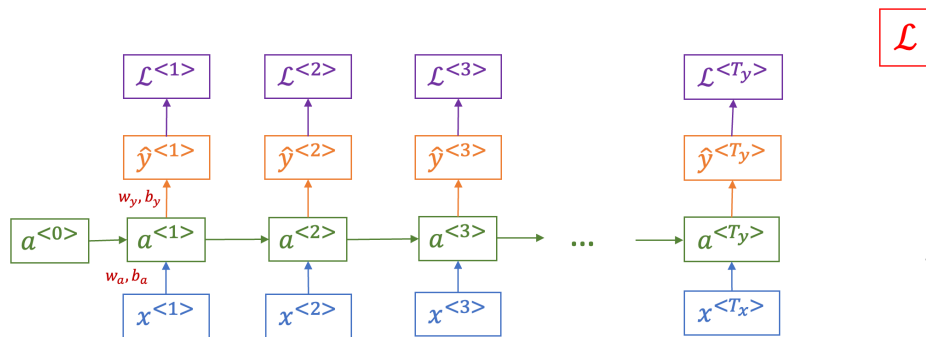
*Backpropagation
thru time*

$$\mathcal{L}^{<t>} = \mathcal{L}(\hat{y}^{<t>}, y^{<t>}) = -y^{<t>} \log(\hat{y}^{<t>}) - (1 - y^{<t>}) \log(1 - \hat{y}^{<t>})$$

$$\mathcal{L}(\hat{y}, y) = \sum_{t=1}^{T_y} \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$$

Sequence models: RNN

- Forward propagation and backpropagation



$$\mathcal{L}^{<t>} = \mathcal{L}(\hat{\mathbf{y}}^{<t>}, \mathbf{y}^{<t>}) = -y^{<t>} \log(\hat{y}^{<t>}) - (1 - y^{<t>}) \log(1 - \hat{y}^{<t>})$$

$$\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y}) = \sum_{t=1}^{T_y} \mathcal{L}(\hat{\mathbf{y}}^{<t>}, \mathbf{y}^{<t>})$$

Backpropagation
thru time

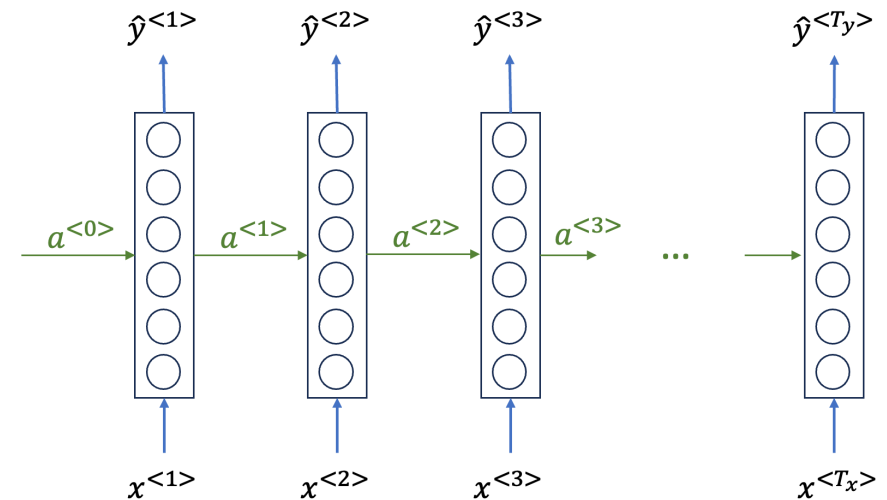
Applying gradient descent is straightforward:

The derivative of the global loss $\mathcal{L}$ w.r.t. a parameter $\mathbf{w}$ = The sum of the derivatives of the losses at each time step $\mathcal{L}^{<t>}$ w.r.t. $\mathbf{w}$.

$$\frac{d\mathcal{L}(\hat{\mathbf{y}}, \mathbf{y})}{d\mathbf{w}} = \sum_{t=1}^{T_y} \frac{d\mathcal{L}(\hat{\mathbf{y}}^{<t>}, \mathbf{y}^{<t>})}{d\mathbf{w}}$$

Sequence models: Different types of RNN

- In the model seen so far: $T_x = T_y$
- Suitable for many applications:
 - Name entity detection.
 - Text segmentation.
 - Audio segmentation: speech/music/...
 - ...
- But for many other applications: $T_x \neq T_y$
 - Translation.
 - Activity recognition in videos.
 - Sentiment analysis.
 - Video captioning.
 - ...

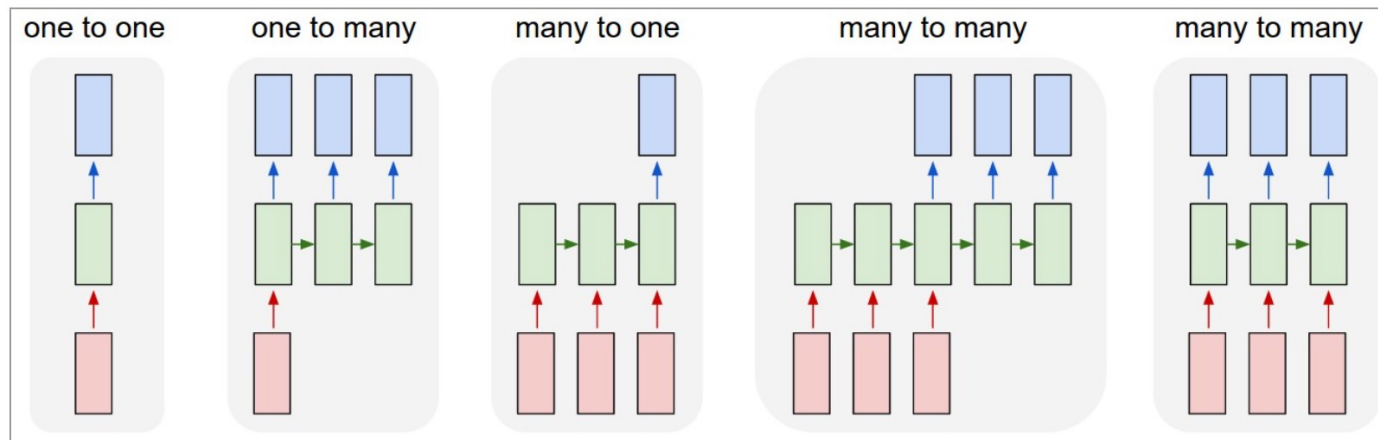


Sequence models: Different types of RNN

- The basic RNN architecture has thus to be adapted to cope with other sequence-based applications.
- A very good reference for this part: The *Andrej Karpathy* blog.

“The Unreasonable Effectiveness of Recurrent Neural Networks”, May 21, 2015.

<https://karpathy.github.io/2015/05/21/rnn-effectiveness/>



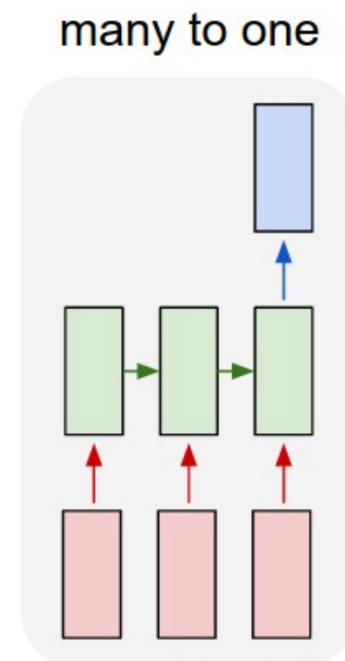
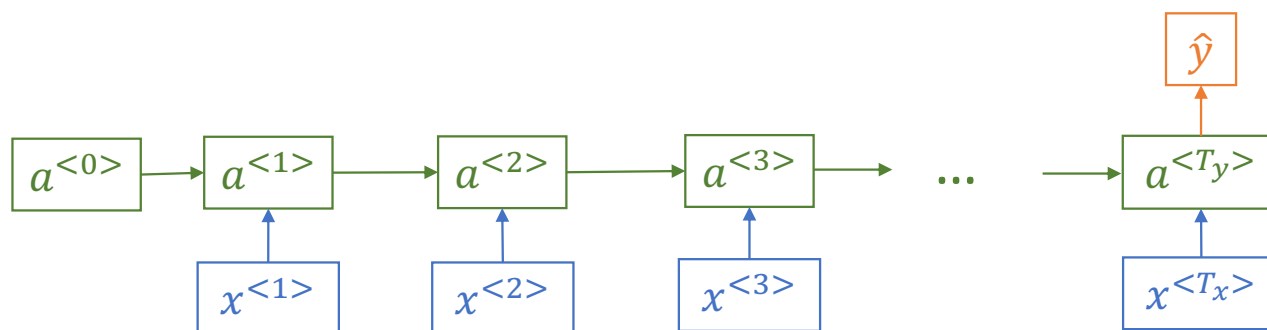
Sequence models: Different types of RNN

- The many-to-one architecture:

Example: sentiment classification.

Input (x): a comment, a sequence of words of different lengths.

Output (y): a rate (1-5) or a binary output (positive/negative).



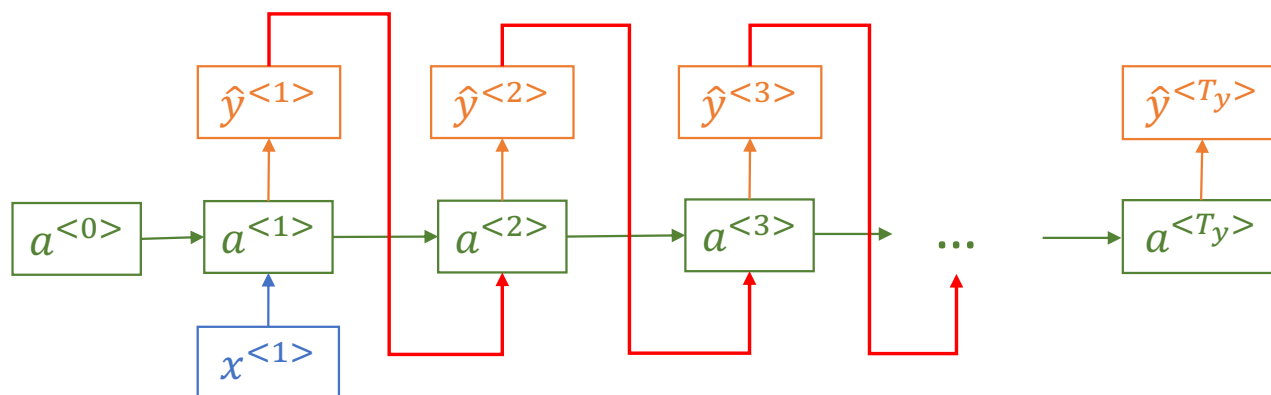
Sequence models: Different types of RNN

- The one-to-many architecture

Example: Music generation.

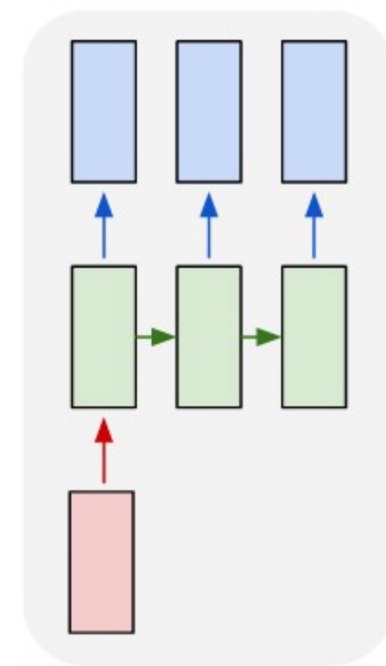
Input (x): random.

Output (y): a sequence of music (a set of notes).



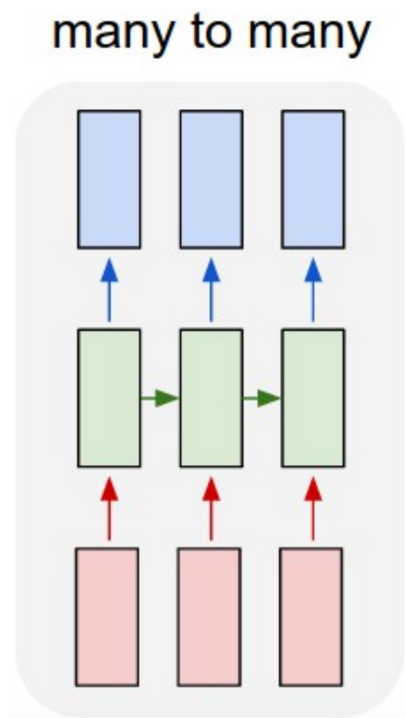
In sequence generation application, $\hat{y}^{<t>}$ is used as input $\hat{x}^{<t+1>}$

one to many

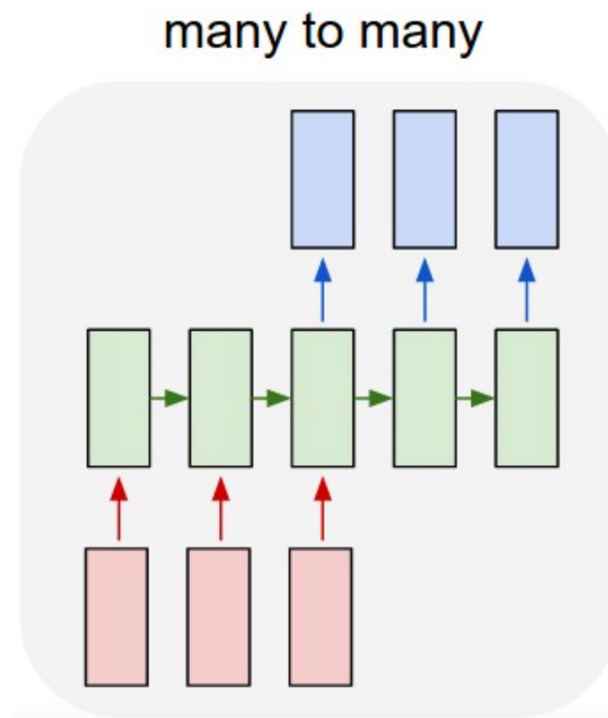


Sequence models: Different types of RNN

- The many-to-many architecture



$$T_x = T_y$$

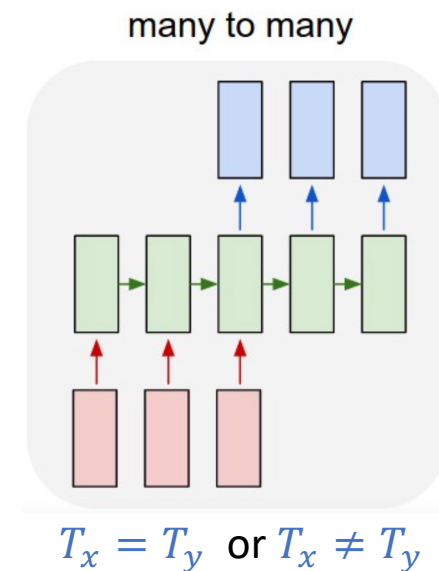
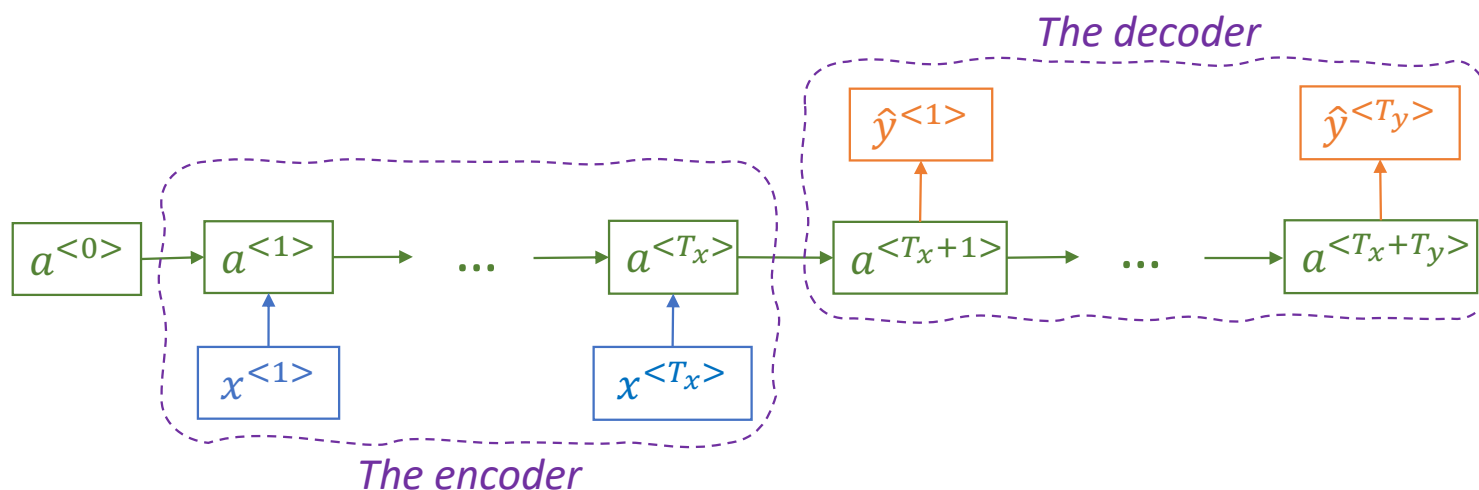


$$T_x = T_y \text{ or } T_x \neq T_y$$

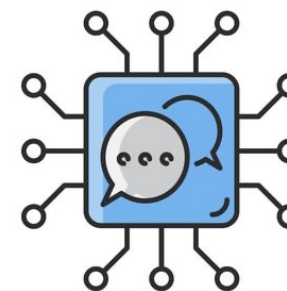
Sequence models: Different types of RNN

- The many-to-many architecture

Example: translation, video captioning

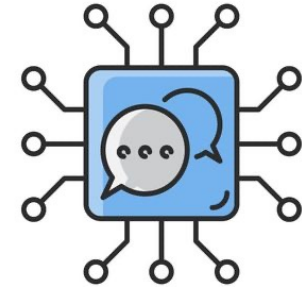


Sequence models: RNN

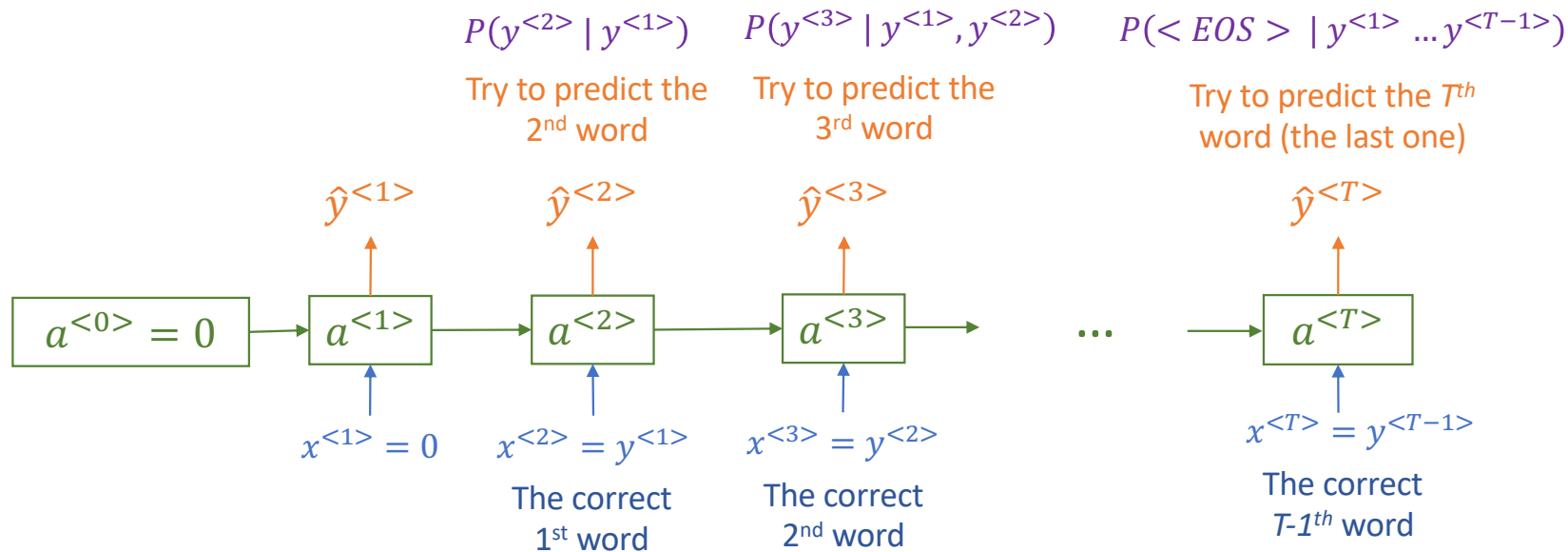


- Building a language model using RNN
 - $P(\text{sentence})$
 - $P(\text{word} \mid \text{previous sentence})$
 - A language model is learned from a training set: A large text.
 - The training text is tokenized:
 - Extract the vocabulary
 - Add **<EOS>** token: end of sentence.
 - Add **<UNK>** token: unknown word. Used during test phase, if we encounter a word that is out of the training vocabulary.

Sequence models: RNN



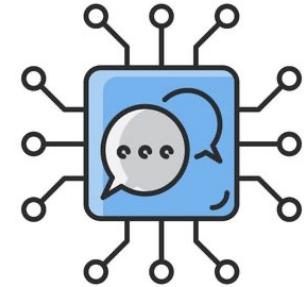
- Building a language model using RNN



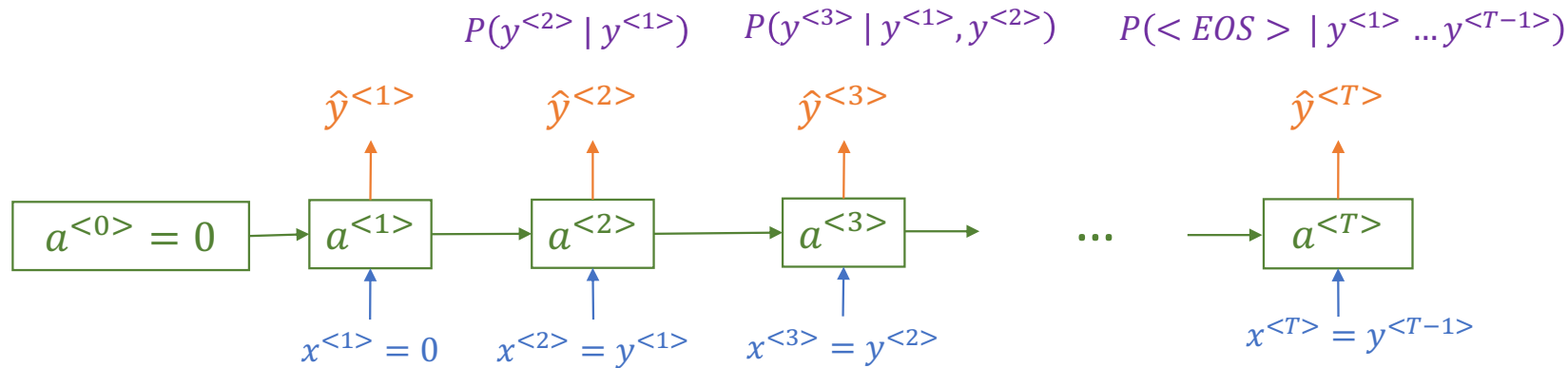
$\hat{y}^{<1>}$: Probability of any word in dictionary. Output of a softmax layer whose dimension = size of the vocabulary.

$\hat{y}^{<1>} = [P(a), \dots, P(course), \dots, P(the), \dots, P(Zyzzzyva), P(< UNK >), P(< EOS >)]$

Sequence models: RNN



- Building a language model using RNN



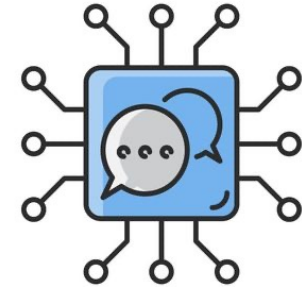
This model learns to predict one word at a time.

Each word is predicted using previous words in the sentence.

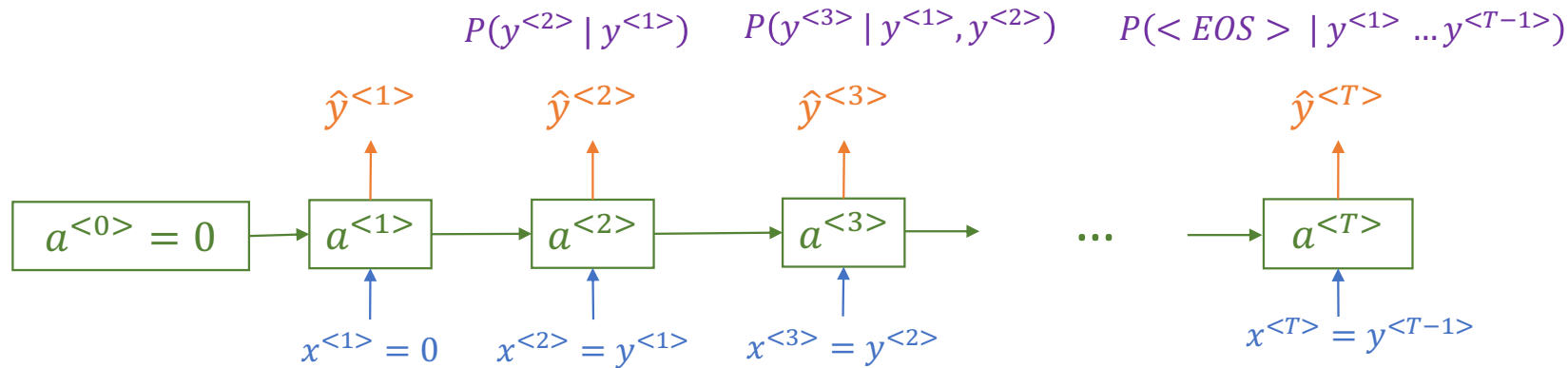
The loss function (softmax loss):

$$\mathcal{L}(\hat{y}^{<t>}, y^{<t>}) = - \sum_i y^{<t>}_i \log(\hat{y}_i^{<t>})$$
$$\mathcal{L} = \sum_t \mathcal{L}(\hat{y}^{<t>}, y^{<t>})$$

Sequence models: RNN



- Making use of an RNN-based language model



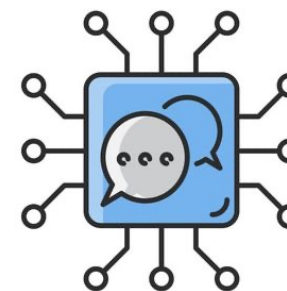
Estimating the probability of a sentence $S = (w_1, w_2, \dots, w_T)$.

- At $t=1$: $P(w_1)$ is given by $\hat{y}^{<1>}$
- At $t=2$: $x^{<2>} = w_1$. The probability $P(w_2 | w_1)$ is given by $\hat{y}^{<2>}$
- At $t=3$: $x^{<3>} = w_2$. The probability $P(w_3 | w_1, w_2)$ is given by $\hat{y}^{<3>}$
- ...

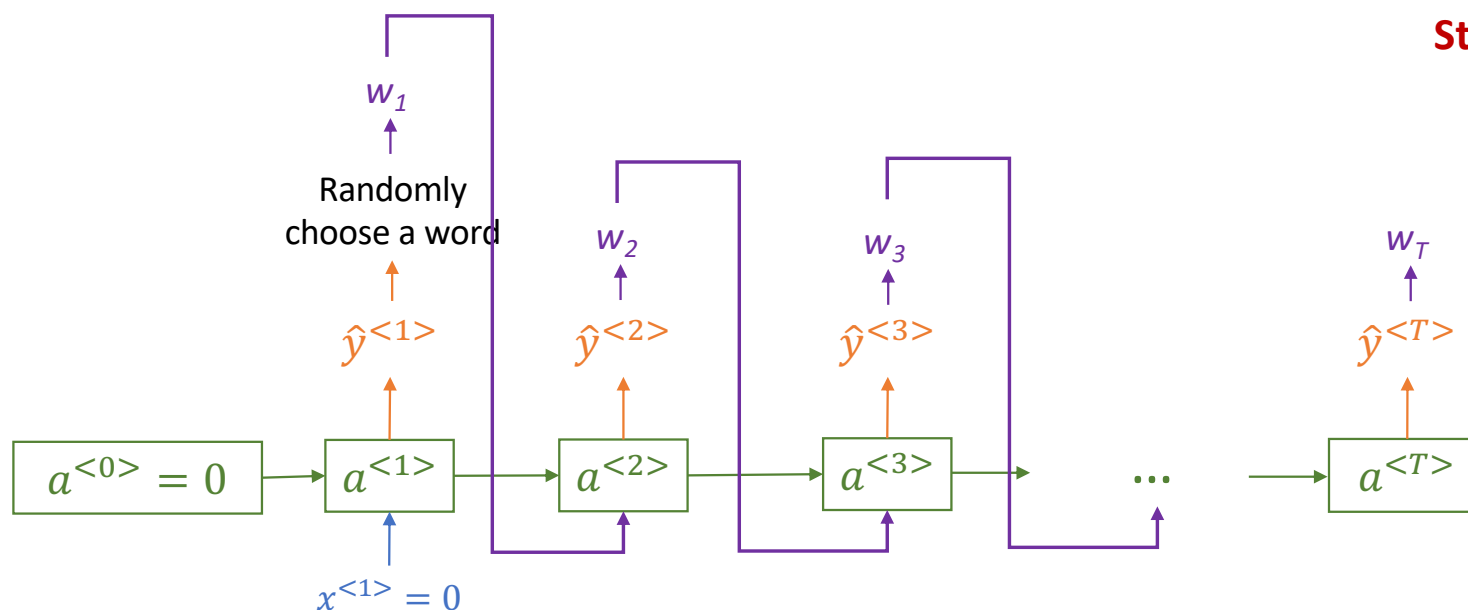
$$P(S) = P(w_1) \cdot P(w_2 | w_1) \cdot P(w_3 | w_1, w_2)$$

Sequence models: RNN

- Making use of an RNN-based language model



Sampling a sentence from the model



Stop when :
T = target number,
or $w_t = \text{<EOS>}$.

Sequence models: RNN

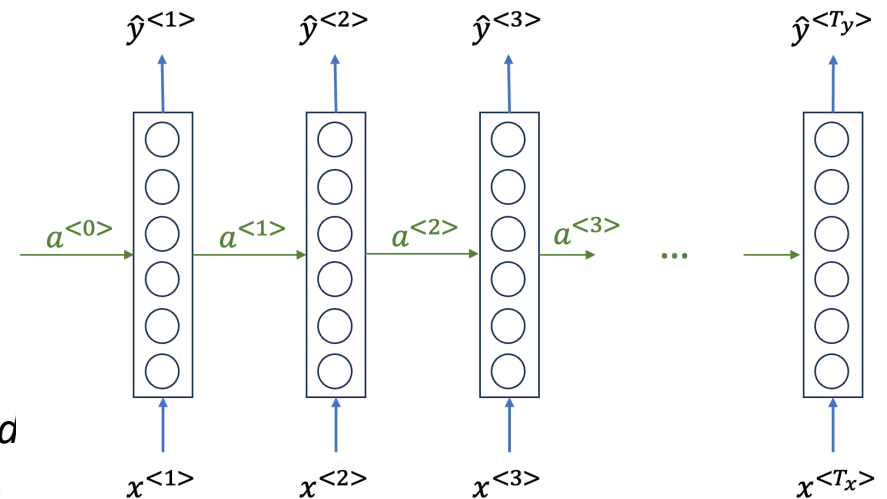
- The vanishing gradient problem

The length of the sequence $\equiv$ The depth of the NN

“The professor announced that the paper/papers submitted by the student, which had been reviewed thoroughly by the committee, was/were now ready to be published.”

To correctly predict “was/were” w.r.t. “paper/papers”, the RNN must remember a long past context.

=> GRU, LSTM...



- Due to vanishing/exploding gradient, a simple RNN cannot do this effectively.
- It can only remember very few past words.
- Simple RNN models only local influences.